

Desafío 1 Sweet Crush

Informe preliminar: análisis y diseño

Oscar Mauricio Trujillo Samboní

1. El problema

El desafío consiste en programar en C++ una versión de consola del juego Sweet Crush. El tablero tiene F filas y C columnas con fichas de seis tipos A a F. En cada jugada el usuario elige una posición esa ficha desaparece, las de arriba caen y aparecen fichas nuevas al azar. Si quedan tres o más fichas iguales seguidas en una fila o columna se eliminan solas y si eso forma nuevas combinaciones se siguen eliminando como cascadas. También se pueden agregar o quitar filas y columnas en cualquier posición durante la partida.

Lo difícil no es el juego sino cómo se guarda el tablero, no podemos usar struct, class, template, string, STL ni funciones de C. Todo el tablero va en un solo bloque unsigned char* de memoria dinámica, cada ficha ocupa exactamente 3 bits y la memoria reservada tiene que corresponder al tamaño real del tablero. Al quitar filas o columnas solo se pide un bloque nuevo si lo usado baja del 65 % de lo reservado.

2. Representación del tablero

Con 3 bits hay 8 valores posibles. Usamos seis para las fichas, uno para las casillas vacías y dejamos el último libre como en la siguiente tabla.

Código	000	001	010	011	100	101	110	111
Ficha	A	B	C	D	E	F	Vacía	Sin uso

El tablero ocupa $(3 * F * C + 7) \gg 3$ bytes, que es dividir los bits entre 8 redondeando hacia arriba. Para la ficha de la fila f y columna c calculamos $\text{bit} = (f * C + c) * 3$, y de ahí sacamos el byte ($\text{bit} \gg 3$) y la posición dentro del byte ($\text{bit} \& 7$). Cuando esa posición es 6 o 7 la ficha queda partida entre dos bytes por ejemplo, la tercera ficha empieza en el bit 6 del byte 0 y termina en el bit 0 del byte 1. Las funciones leerFicha y escribirFicha manejan ese caso con máscaras y desplazamientos y ya las probamos con varios tamaños de tablero.

3. Diseño del programa

El código está dividido en tres partes. tablero.h/.cpp tiene todo lo de memoria y bits, crear el tablero, leer y escribir fichas, llenarlo al azar y mostrarlo en letras, en binario y byte por byte. juego.h/.cpp tendrá la lógica de las jugadas y main.cpp el menú, eliminar ficha, agregar o quitar fila o columna y ver estadísticas. Como no se puede usar struct el estado del juego son variables de main que se pasan por referencia.

Jugada y gravedad: La ficha elegida se marca como vacía, luego cada columna se recorre de abajo hacia arriba con dos índices, uno que lee y otro que escribe, bajando las fichas que no están vacías, las casillas que quedan arriba se llenan al azar.

Combinaciones y cascada: Usé una máscara temporal de 1 bit por casilla, recorreremos filas y columnas contando fichas iguales seguidas y marcamos los tramos de 3 o más así una ficha que está en una combinación horizontal y en una vertical se elimina una sola vez, se vacían las marcadas, se aplica gravedad, se rellena y se libera la máscara. Esto se repite hasta que no haya más combinaciones, y cada vuelta cuenta como una cascada.

Filas y columnas. Si al agregar el tablero no cabe en lo reservado, pedimos un bloque exacto con `new[]`, copiamos las fichas ya reubicadas y liberamos el viejo con `delete[]`. Si cabe movemos las fichas dentro del mismo bloque empezando por el final para no pisar ninguna. Al quitarcorremos las fichas hacia adelante y solo pedimos un bloque más pequeño si los bytes necesarios por 100 dan menos que la capacidad por 65, que es la regla del 65 % sin usar decimales. Después de cualquier cambio se revisan combinaciones y cascadas.

Puntaje. Propongo 10 puntos por cada ficha eliminada en una combinación, multiplicados por el número de la cascada ($\times 1$, $\times 2$, ...). La ficha que quita el usuario no suma. También llevamos fichas eliminadas, combinaciones y cascadas.